University of Calgary
Department of Electrical and Software Engineering

ENSF 694

Data Structures and Algorithms

Campus Navigation and Event Management System

Technical Report

Course: ENSF 694 – Data Structures and Algorithms
Term: Summer 2026
Group: 4
Members: Muhammad Ibrahim Khan
Saman Pordanesh
Instructor: Maan Khedr

August 2026

Contents

1 Introduction **2**

2 Design Decisions **2**
2.1 Campus Map and Shortest Path 2
2.2 Navigation History 2
2.3 Room and Event Bookings 2
2.4 Priority-Based Service Requests 2
2.5 Building and Resource Lookup 3
2.6 Incoming Request Processing 3

3 Complexity Analysis **3**

4 Demo Scenarios **4**
4.1 Shortest Path Queries 4
4.2 Navigation Undo 5
4.3 Booking Range Query 6
4.4 Priority Queue 7
4.5 Fast Resource Lookup 8
4.6 Incoming Request Pipeline 9

5 Challenges and Lessons Learned **9**

6 AI Use Disclosure **9**

7 Contributions **10**

1 Introduction

We built a Campus Navigation and Event Management System to handle common campus tasks. It supports shortest-path navigation, navigation history and undo, room bookings, priority-based service requests, fast resource lookup, and FIFO processing of incoming requests.

Our goal was to match each task with a suitable data structure without making the application unnecessarily complicated. The result is a C++ console application that loads the campus network from a CSV file.

2 Design Decisions

2.1 Campus Map and Shortest Path

We store the campus as a **weighted adjacency matrix**. Each building is a vertex and each non-zero matrix entry represents the walking time between two connected buildings. The test map contains at least 15 buildings and 25 pathways.

We chose an adjacency matrix because the campus is small and fixed, and the matrix gives direct access to the weight between any two buildings. An adjacency list would use less space for a sparse graph, but would add extra traversal logic without providing much benefit at this scale.

The program computes shortest routes with **Dijkstra's algorithm**, which works here because all walking-time weights are non-negative. Breadth-first search was not used because it does not account for different edge weights.

2.2 Navigation History

We store up to 10 routes in a fixed array and use it as a **stack**. New routes are added to the top, and undo removes the most recent route.

This directly matches the last-in, first-out behavior of undo. A queue would instead remove the oldest route first.

2.3 Room and Event Bookings

We keep bookings in a **sorted array** ordered by day and starting time. A binary search locates the correct position for each booking. The same ordering makes it easier to find the next event and process time-range queries.

An unsorted array would make insertion simpler, but ordered and range queries would require searching through all bookings. The current design keeps the bookings ordered at the cost of shifting elements during insertion and deletion.

2.4 Priority-Based Service Requests

The service queue stores requests in an array with one of three priorities: Emergency, Standard, or Low. New requests go at the end of the array. When serving a request, the program scans the array for the highest-priority item.

We also considered a binary heap, which would provide faster removal for a larger queue. We kept the array because the queue is limited to 100 requests and only has three priority levels.

2.5 Building and Resource Lookup

For fast lookup, we use a **hash table** with 257 slots. Building and room IDs act as keys. The table handles collisions using **linear probing**, and deleted entries are marked so later searches can continue through the probe sequence.

We considered a search tree because it provides logarithmic lookup and ordered traversal. However, this component does not need ordering, so a hash table is a better match for direct lookup.

2.6 Incoming Request Processing

We place incoming navigation and service requests in a **circular FIFO queue**. Its front and rear indices move through a fixed array of 100 positions.

This allows both enqueue and dequeue operations without shifting the remaining elements. A stack was not suitable because requests must be processed in arrival order.

3 Complexity Analysis

Let V be the number of campus buildings and n the number of elements stored in the corresponding structure.

Operation	Best	Average	Worst	Space
Campus graph	—	—	—	$O(V^2)$
Shortest path	$O(V^2)$	$O(V^2)$	$O(V^2)$	$O(V)$
History push / undo	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Booking insertion	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Booking lookup	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$
Booking range query	$O(\log n)$	$O(\log n + r)$	$O(n)$	$O(n)$
Service request add	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Serve highest priority	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Resource lookup	$O(1)$	$O(1)$	$O(n)$	$O(n)$
Request enqueue / dequeue	$O(1)$	$O(1)$	$O(1)$	$O(n)$

Table 1: Complexity of the main system operations. Here, r is the number of bookings returned by a range query.

The adjacency matrix requires $O(V^2)$ storage. The implemented version of Dijkstra's algorithm searches the matrix and linearly selects the next closest vertex, resulting in $O(V^2)$ running time.

Booking insertion and deletion may require shifting array elements. Hash-table operations are constant time on average but can become linear if many collisions occur. Stack and circular-queue operations use direct array access and require constant time.

4 Demo Scenarios

The screenshots below cover the six required application scenarios and show the data structure used by each component.

4.1 Shortest Path Queries

Figure 1: Science A to MFH is the first shortest-path example. The displayed route and total walking time show the result from Dijkstra's algorithm; a second query, from ICT to Residence, appears in Figure 2.

4.2 Navigation Undo

```
Navigation
1. Find route
2. Undo route
3. Previous routes
4. Back
Choice: 1

Where are you starting?
1. Library
2. Science A
3. ICT
4. ENG Block
5. Gym
6. Student U
7. Parkade
8. MFH
9. Residence
10. Arts
11. Business
12. Dining
13. Health
14. Admin
15. Research
Selection: 3

Where are you going?
1. Library
2. Science A
3. ICT
4. ENG Block
5. Gym
6. Student U
7. Parkade
8. MFH
9. Residence
10. Arts
11. Business
12. Dining
13. Health
14. Admin
15. Research
Selection: 9
Route: ICT -> Student U -> Residence
Travel time: 8 minutes

Navigation
1. Find route
2. Undo route
3. Previous routes
4. Back
Choice: 2
Back to ICT

Navigation
1. Find route
2. Undo route
3. Previous routes
4. Back
```

```
Navigation
1. Find route
2. Undo route
3. Previous routes
4. Back
Choice: 2
Back to ICT

Navigation
1. Find route
2. Undo route
3. Previous routes
4. Back
Choice: 3
1. Library -> ICT | 4 minutes
2. Science A -> MFH | 4 minutes
```

Figure 2: These screenshots show a route from ICT to Residence followed by an undo operation. Undo restores ICT as the previous origin, while the remaining route history stays in the Navigation History component.

4.3 Booking Range Query

```
Main menu
1. Navigation
2. Bookings
3. Service desk
4. Resources
5. Incoming requests
6. Run tests
7. Exit
Choice: 2

Bookings
1. Add booking
2. Remove booking
3. Find booking
4. Events in time range
5. Next event
6. Events on a day
7. Back
Choice: 4
Day number: 0
Start of range: 10
End of range: 14
LIB101 | Day 0 | 10:00 - 11:00 | Event 10
SCI301 | Day 0 | 11:00 - 12:00 | Event 11
ICT101 | Day 0 | 12:00 - 13:00 | Event 12
ICT102 | Day 0 | 13:00 - 14:00 | Event 13
```

Figure 3: A Day 0 query between 10:00 and 14:00 returns the four events that overlap the requested interval. The result shows how the Booking component uses its sorted structure for a range query.

4.4 Priority Queue

```
Service desk
1. Add request
2. Serve next
3. Back
Waiting requests: 1
Choice: 1
Describe the problem: Net down

Urgency:
1. Emergency
2. Standard
3. Low
Selection: 1
Emergency request added.

Service desk
1. Add request
2. Serve next
3. Back
Waiting requests: 2
Choice: 1
Describe the problem: Need markers

Urgency:
1. Emergency
2. Standard
3. Low
Selection: 2
Standard request added.

Service desk
1. Add request
2. Serve next
3. Back
Waiting requests: 3
Choice: 2

Now serving:
Emergency - Net down

Service desk
1. Add request
2. Serve next
3. Back
Waiting requests: 2
Choice: 2

Now serving:
Standard - Need markers

Service desk
1. Add request
2. Serve next
3. Back
Waiting requests: 1
Choice: 2

Now serving:
Low - Broken Projector
```

Figure 4: The Service Desk receives Emergency, Standard, and Low-priority requests. It serves them by urgency rather than arrival order, with the highest-priority request selected first.

4.5 Fast Resource Lookup

```
Main menu
1. Navigation
2. Bookings
3. Service desk
4. Resources
5. Incoming requests
6. Run tests
7. Exit
Choice: 4

Resources
1. Find
2. Add
3. Remove
4. Back
Stored records: 20
Choice: 1
Resource ID: ICT
ICT | ICT | Building
Table checks: 1

Resources
1. Find
2. Add
3. Remove
4. Back
Stored records: 20
Choice: 1
Resource ID: Something
Not found.
```

Figure 5: The Resource Lookup finds the ICT record and then reports a missing key. Both searches use the unique ID as the hash-table key and follow the same linear-probing logic.

4.6 Incoming Request Pipeline

```
Main menu
1. Navigation
2. Bookings
3. Service desk
4. Resources
5. Incoming requests
6. Run tests
7. Exit
Choice: 5

Incoming requests
1. Add navigation request
2. Add service request
3. Process next
4. Test 20 requests
5. Back
Waiting requests: 0
Choice: 4

Processing order:
1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15 -> 16 -> 17 -> 18 -> 19 -> 20
All 20 requests were processed in arrival order.
```

Figure 6: All 20 numbered requests leave the Incoming Request component in the same order in which they entered. This confirms that the circular FIFO queue preserves arrival order.

5 Challenges and Lessons Learned

The most error-prone part was keeping the different array-based structures in the correct order. Bookings must stay sorted after insertion and deletion, while navigation history and incoming requests use different removal orders.

Hash-table deletion also required care because removing an entry cannot break the probing sequence for later entries. Using separate "used" and "active" states solved this problem.

The project made the trade-offs between these structures clear. Direct lookup, ordered events, undo, routing, priority processing, and FIFO processing each need a different access pattern.

6 AI Use Disclosure

We used AI tools for support during the final stages of the project. They helped us develop ideas for requirement-based test cases, testing code, and were used to generate all of the test data. AI was also used to review the complexity analysis and improve the wording and organization of this report. We reviewed each suggestion and adjusted it to match the actual

A representative prompt used for testing was:

> "Based on the project requirements and the current C++ implementation, generate simple test cases for each required feature. Include tests for the 15-node/25-edge campus map, shortest path and undo, at least 100 bookings, three service priorities, successful and unsuccessful hash-table lookups, and 20 FIFO incoming requests. Keep the testing code simple and consistent with the existing classes."

We checked the resulting test data and code against the implemented system before using them. We also used AI to review the Big-O descriptions and rephrase parts of the report for clarity and conciseness.

7 Contributions

Group Member	Primary Contributions
Muhammad Ibrahim Khan	Class prototypes, class function implementation, and report writing.
Saman Pordanesh	Main function and test files, README file, report writing, and execution of the demo scenarios.

Table 2: Group member contributions.